

Cet examen est composé de 3 exercices indépendants qui peuvent être traités dans un ordre quelconque. Il est donc conseillé de lire l'énoncé dans sa totalité avant de commencer.

Pour les questions commençant par "*Proposer une solution*" vous devez expliquer textuellement comment fonctionne votre solution, pour les questions commençant par "*Donner l'algorithme*" vous devez donner l'algorithme, avec des commentaires.

Pour les algorithmes, vous ne disposez pas d'autres objets/fonctions que des listes, files ou piles et leurs fonctions associées définies dans le cours.

Attention, la note sera diminuée si :

- Le soin apporté à la rédaction et à la copie sont insuffisants,
- Toutes les réponses ne sont pas justifiées par un raisonnement clair et précis.

Exercice 1 : Couper un graphe (9 points)

Soit un graphe $G(V, E)$ non orienté connexe, codé dans un tableau de listes de voisins LV . L'objectif de cet exercice est d'obtenir, à partir de G , deux sous-graphes G_1 et G_2 , équilibrés en nombre d'arêtes : la différence du nombre d'arêtes entre les deux composantes doit être inférieure au degré maximal du graphe G . Les deux sous-graphes n'ont pas forcément besoin d'être connexes (le problème est NP-Complet sinon) mais la solution proposée doit favoriser cette propriété : lorsque vous avez le choix entre deux sommets il faut prendre un sommet connecté au sous-ensemble.

Question 1.1 : *Proposer une solution pour calculer le nombre d'arêtes d'un sous-graphe du graphe G donné dans une liste de sommets sg .*

Question 1.2 : *Donner l'algorithme d'une fonction qui implémente cette proposition.*

Question 1.3 : *Proposer une solution pour trouver un sommet de la bordure d'une liste de sommets sg parmi une liste de sommets sl du graphe G .*

Question 1.4 : *Donner l'algorithme d'une fonction qui implémente cette proposition. La fonction retourne un sommet quelconque si aucun sommet de la liste sl n'appartient à la bordure de la liste sg .*

Question 1.5 : *Proposer une solution permettant de réaliser le découpage d'un graphe G en sous-graphes SG_1 et SG_2 de telle sorte que le nombre des arêtes dans les deux composantes soit aussi équilibré que possible.*

Question 1.6 : *Donner l'algorithme qui implémente cette proposition pour calculer les sous-graphes, donnés par les listes $sg1$ et $sg2$.*

Correction Exercice 1 : Couper un graphe

Solution question 1.1 : *Proposer une solution pour calculer le nombre d'arêtes d'un sous-graphe sg du graphe G donné dans une liste de sommets.*

Pour calculer le nombre d'arêtes d'une liste de sommets sg , il faut, pour chaque élément de la liste vérifier si les voisins, tels qu'il sont définis dans la structure LV , sont eux-mêmes dans la liste sg . Attention, en faisant ce compte, chaque arête est compté deux fois, il faut donc diviser le résultat par deux.

Solution question 1.2 : *Donner l'algorithme qui implémente cette proposition.*

L'algorithme 1 donne la fonction `nbAretesSG`.

Algorithme 1 : Fonction donnant le nombre d'arêtes d'une liste de sommets sg pour un graphe codé dans une liste de voisins LV

Entrées : $LV[0..N-1]$: tableau des listes de voisins
 sg : liste des sommets du sous-graphe

1 **Fonction** $nbAretesSG(LV, sg)$:
Données : $nbAretes$: entier , **init** à 0

2 **début**

3 **pour** $i \leftarrow 0$ à $sg.size() - 1$ **faire**

4 $som \leftarrow sg.get(i)$

5 **pour** $j \leftarrow 0$ à $LV[som].size() - 1$ **faire**

6 **si** $sg.contains(LV[som].get(j))$ **alors**

7 $nbAretes \leftarrow nbAretes + 1$

8 **retourner** $nbAretes/2$

Solution question 1.3 : Proposer une solution pour trouver un sommet de la bordure d'une liste de sommets sg parmi une liste de sommets sl du graphe G .

Pour trouver un sommet de la bordure d'une liste de sommets sg parmi une liste de sommets du graphe G , il faut parcourir la liste de sommets sg et pour chacun de leur voisin vérifier s'il fait partie de la liste sl sans faire partie de la liste sg .

Solution question 1.4 : Donner l'algorithme d'une fonction qui implémente cette proposition. La fonction retourne un sommet quelconque si aucun sommet de la liste n'appartient à la bordure de la liste sg

L'algorithme 2 donne la fonction *trouveSommet*.

Algorithme 2 : Fonction donnant un sommet de la bordure d'une liste de sommets sg parmi une liste de sommets sl ou un sommet quelconque si aucun sommet de la bordure n'appartient à la liste sl .

Entrées : $LV[0..N-1]$: tableau des listes de voisins
 sg, sl : listes des sommets

1 **Fonction** $trouveSommet(LV, sg, sl)$:
Données : som, i : entiers , **init** à 0
 $trouve$: booléen, **init** à faux

2 **début**

3 **tant que** $(\neg trouve) \wedge (i < sg.size())$ **faire**

4 $som \leftarrow sg.get(i)$

5 **pour** $j \leftarrow 0$ à $sl.size() - 1$ **faire**

6 $vois \leftarrow sl.get(j)$

7 **si** $(\neg sg.contains(vois)) \wedge (LV[som].contains(vois))$ **alors** $trouve \leftarrow true$

8 **si** $\neg trouve$ **alors** $i \leftarrow i + 1$

9 **si** $\neg trouve$ **alors** $vois \leftarrow sl.get(0)$

10 **retourner** $vois$

Solution question 1.5 : Proposer une solution permettant de réaliser le découpage d'un graphe

G en sous-graphes SG_1 et SG_2 de telle sorte que le nombre des arêtes dans les deux composantes soit aussi équilibré que possible.

Pour réaliser cette coupe, il faut constituer deux listes $sg1$ et $sg2$. On part d'une liste des sommets et à chaque itération on ajoute un sommet, obtenu avec la fonction *trouveSommet*, à la liste qui a le moins d'arêtes. Le nombre d'arêtes est, bien sûr calculé avec la fonction *nbAretes*. Le sommet ajouté à la liste amène forcément un nombre d'arêtes inférieur ou égal au degré maximal du graphe à la liste. Puisque le sommet est ajouté à la liste ayant le plus petit nombre d'arêtes, alors la différence entre les deux sommets reste inférieure au degré maximal du graphe. Le sommet est retiré de la liste et on itère jusqu'à ce qu'il n'y ait plus de sommet libre.

Solution question 1.6 : Donner l'algorithme qui implémente cette proposition pour calculer les sous-graphes, donnés par les listes $sg1$ et $sg2$

L'algorithme 3 donne la fonction *coupe*.

Algorithme 3 : Fonction coupant un graphe en deux composantes équilibrées en nombre d'arêtes

Entrées : $LV[0..N-1]$: tableau des listes de voisins

1 Fonction *coupe*(LV) :

Données : $g1, g2, sommetsLibres$: listes, **init à** $\emptyset$

2 début

3 **pour** $i \leftarrow 0$ à $N-1$ **faire** $sommetsLibres.add(i)$

4 **tant que** $sommetsLibres.peek() \neq NULL$ **faire**

5 **si** $nbAretesSG(g1) \leq nbAretesSG(g2)$ **alors**

6 $som \leftarrow trouveSommet(LV, g1, sommetsLibres)$

7 $g1.add(som)$

8 **sinon**

9 $som \leftarrow trouveSommet(LV, g2, sommetsLibres)$

10 $g2.add(som)$

11 $sommetsLibres.remove(som)$

12 **retourner** $g1, g2$

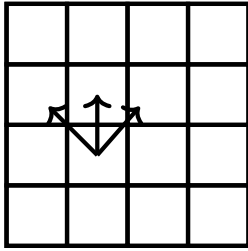
Exercice 2 : Parcours de damier (7 points)

FIGURE 1 – Passages d'une case à une autre

Soit un damier carré de côté N . Nous souhaitons parcourir ce damier, de case en case, du bord inférieur ($i = 1$) au bord supérieur ($i = N$). Les seuls déplacements autorisés sont les passages vers les cases adjacentes sur la ligne supérieure, comme cela est montré sur la figure 1.

Lorsque nous passons de la case (i, j) à la case (i', j') nous obtenons un nombre de points $Gain((i, j), (i', j'))$, dépendant de la case de départ (i, j) et la case d'arrivée (i', j') .

L'objectif est de réaliser ce parcours en obtenant le plus grand nombre de points.

Question 2.1 : Donner la structure de la solution optimale en fonction des sous-problèmes

Question 2.2 : Définir récursivement la solution optimale et montrer que l'application directe de la récursivité suppose le re-calcul de la valeur pour une même case.

Question 2.3 : Donner l'algorithme de calcul d'une solution optimale en programmation dynamique.

Question 2.4 : Quelle est la complexité de cet algorithme ?

Correction Exercice 2 : Parcours de damier

Ce problème est très proche du problème de la pyramide que nous avons donné en exercice puisqu'il s'agit également d'un parcours. Il n'y a cependant pas que la forme du support qui change. Le gain est obtenu en passant d'une case à l'autre et non en fonction de la case d'arrivée. Cela change l'expression de la récurrence.

Solution question 2.1 : Caractériser la structure de la solution optimale

Le gain obtenu pour une case dépend du gain obtenu dans les cases qui peuvent y accéder et du gain de passage de ces cases à la case considérée. La solution optimale à la ligne l est donc la meilleure solution qui puisse être obtenue à partir de la ligne $l - 1$ en ajoutant la fonction $Gain$.

Solution question 2.2 : Définir récursivement la solution optimale et montrer que l'application directe de la récursivité suppose le re-calcul de la valeur pour une même case.

Soit $Damier(l, c)$, la solution optimale à la case (l, c) du damier. La définition récursive de la solution optimale $G^*(l)$ à la ligne l est donc :

$$\begin{cases} G^*(0) = 0 \\ G^*(l) = \max_{1 \leq c \leq N} (Damier(l, c)) \\ Damier(l, c) = \max_j (Damier(l-1, c+j) + Gain((l-1, c+j), (l, c))) \end{cases}$$

avec :

$$\begin{cases} j \in \{-1, 0, 1\} \text{ si } 1 < l < N \\ j \in \{0, 1\} \text{ si } l = 1 \\ j \in \{-1, 0\} \text{ si } l = N \end{cases}$$

Comme nous pouvons le constater dans l'équation, chaque point va intervenir dans le calcul de 3 valeurs. De plus, si nous utilisons directement l'équation de récurrence pour écrire notre programme, le calcul pour une case parcourra l'ensemble de la pyramide inférieure à cette case, de l'ordre $O(3^N)$. Comme nous faisons ces calculs pour chaque case du damier la complexité totale sera de l'ordre de $O(N \times 3^N)$. L'utilisation de la programmation dynamique, avec la mémorisation des valeurs intermédiaires va donc permettre une réduction significative de la complexité puisqu'un seul calcul sera réalisé par case du damier.

Solution question 2.3 : *Donner l'algorithme de calcul d'une solution optimale en programmation dynamique.*

Note : suite à une erreur de manipulation cette question n'était pas présente dans le sujet que vous avez traité. Bien sûr la notation en tient compte. Elle tient compte également des algorithmes qui ont éventuellement été donnés. La question, et sa correction, sont donc ajoutées ici à titre informatif.

Pour réaliser l'algorithme de programmation dynamique nous utilisons un tableau *Damier* à deux dimensions qui donne, pour chaque case le meilleur gain possible. L'algorithme 4 permet de calculer ce tableau pour damier de taille $N \times M$. Il commence par mettre à 0 toutes les valeurs de la première ligne puis cherche, pour chaque case des niveaux suivants, le plus grand gain obtenu à partir des cases de la ligne en dessous. Pour finir, il cherche dans la dernière ligne le plus grand possible.

Solution question 2.4 : *Quelle est la complexité de cet algorithme ?*

Note : compte tenu du fait qu'il manquait la question 3, cette question ne pouvait se comprendre que comme l'évaluation de la complexité de la solution récursive. Pour cela il faut noter que le parcours descendant implique à chaque fois 3 appels récursifs, ce qui fait une complexité en $O(3^N)$.

Pour la solution de programmation dynamique, compte tenu des deux boucles imbriquées de l'algorithme, la complexité est en $O(N^2)$

Algorithme 4 : Algorithme dynamique

```
1  Données :
2    Gain : fonction de gain
3    N : nombre de lignes du damier
4    M : nombre de colonnes du damier
5  Fonction damierDynamique()
6    Entier l : ligne, initialisé à 1
7    Entier c : colonne
8    Entier gainCase : meilleur gain depuis les cases adjacentes
9    Entier gainTotal : gain total
10   Tableau Damier[] : tableau des gains par cases
11  début
12    pour c de 1 à M faire Damier[0][c]  $\leftarrow$  0
13    pour l de 2 à N faire
14      pour c de 1 à M faire
15        gainCase  $\leftarrow$  Damier[l - 1][c] + Gain((l - 1, c), (l, c))
16        si c > 1 alors
17          tmp  $\leftarrow$  Damier[l - 1][c - 1] + Gain((l - 1, c - 1), (l, c))
18          if tmp > gainCase then gainCase  $\leftarrow$  tmp
19        si c < M alors
20          tmp  $\leftarrow$  Damier[l - 1][c + 1] + Gain((l - 1, c + 1), (l, c))
21          if tmp > gainCase then gainCase  $\leftarrow$  tmp
22        Damier[l][c]  $\leftarrow$  gainCase
23    gainTotal  $\leftarrow$  0
24    pour c de 1 à M faire
25      si gainTotal < Damier(N, c) alors gainTotal  $\leftarrow$  Damier(N, c)
26  retourner gainTotal
```

Exercice 3 : Tablettes de chocolat (4 points)

Une chocolatière souhaite produire des tablettes de chocolat. La recette qu'elle utilise nécessite des amandes, du croquant et des noisettes, en plus du cacao. Elle souhaite faire deux types de tablettes, l'une avec 20 grammes d'amandes, 10 grammes de noisettes et 10 grammes de croquant, et l'autre avec 20 grammes de noisettes, 10 grammes d'amandes et 10 grammes de croquant. Les tablettes avec plus d'amandes sont vendues 3 € et les tablettes avec plus de noisettes sont vendues 4 €.

Dans son stock, la chocolatière possède 70 grammes de croquant, 120 grammes d'amandes, 120 grammes de noisettes et suffisamment de cacao pour faire les tablettes.

Question 3.1 : *Écrire le programme linéaire permettant à la chocolatière de trouver le nombre de tablettes de chaque type qui lui permettra de maximiser son profit, en supposant que toutes les tablettes seront vendues.*

Question 3.2 : *Résoudre le problème par la méthode graphique.*

Question 3.3 : *Résoudre le problème par la méthode du simplexe.*

Question 3.4 : *Donner la solution du problème d'optimisation.*

Correction Exercice 3 : Tablettes de chocolat

Solution question 3.1 : *Écrire le programme linéaire permettant à la chocolatière de trouver le nombre de tablettes de chaque type qui lui permettra de maximiser son profit, en supposant que toutes les tablettes seront vendues.* Pour le programme linéaire on pose x_1 le nombre de tablettes de chocolat ayant le plus d'amandes et x_2 celui qui aura le plus de noisettes.

Puisque la chocolatière souhaite maximiser son profit, la fonction objectif dépend du nombre des tablettes et de leur prix : $3x_1 + 4x_2$.

Les contraintes du problèmes sont données par la limitation des amandes, des noisettes et du croquant. La contrainte liée aux amandes est : $20x_1 + 10x_2 \leq 120$, la somme du poids des amandes utilisées pour les tablettes ne doit pas dépasser 120 grammes. La contrainte liée au croquant est : $10x_1 + 10x_2 \leq 70$, la somme du poids du croquant utilisé pour les tablettes ne doit pas dépasser 70 grammes. La contrainte liée aux noisettes est : $10x_1 + 20x_2 \leq 120$, la somme du poids des noisettes utilisées pour les tablettes ne doit pas dépasser 120 grammes.

Ce qui donne le programme linéaire suivant :

$$\left\{ \begin{array}{lll} \text{maximiser} & 3x_1 + 4x_2 & \\ \text{avec les contraintes} & 20x_1 + 10x_2 \leq 120 \text{ (amandes)} & \\ & 10x_1 + 10x_2 \leq 70 \text{ (croquant)} & \\ & 10x_1 + 20x_2 \leq 120 \text{ (noisettes)} & \\ \text{et } x_1, x_2 & \geq 0 & \end{array} \right.$$

Qui se simplifie en :

$$\left\{ \begin{array}{l} \text{maximiser} \quad 3x_1 + 4x_2 \\ \text{avec les contraintes} \quad x_1 + 2x_2 \leq 12 \\ \quad \quad \quad x_1 + x_2 \leq 7 \\ \quad \quad \quad 2x_1 + x_2 \leq 12 \\ \text{et } x_1, x_2 \geq 0 \end{array} \right.$$

Solution question 3.2 : Résoudre le problème par la méthode graphique.

Pour résoudre par la méthode graphique il est nécessaire de tracer les droites suivantes :

- $y = 12 - 2x$ pour la contrainte sur les amandes
- $y = -x + 7$, pour la contrainte sur le croquant
- $y = \frac{1}{2}x + 6$, pour la contrainte sur les noisettes

Pour optimiser le problème il faut déplacer la droite d'équation $z = 3x_1 + 4x_2$.

La figure 2 donne la résolution graphique du problème. Nous pouvons voir que le meilleur résultat en obtenu au point A avec 2 tablettes avec plus d'amandes et 5 tablettes avec plus de noisettes. Le gain obtenu est $z = 26$;

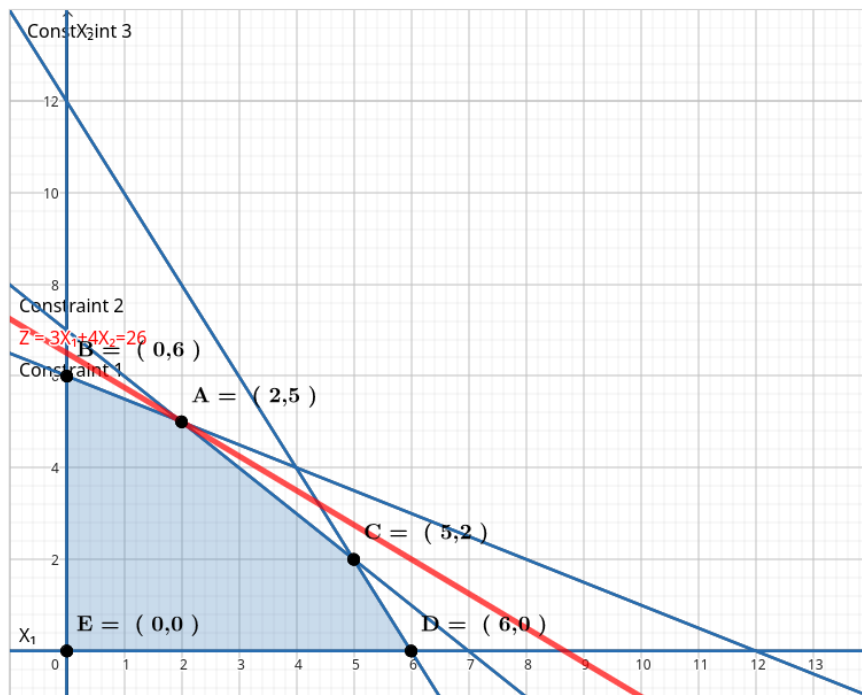


FIGURE 2 – Résolution graphique du programme linéaire des tablettes

Solution question 3.3 : *Résoudre le problème par la méthode du simplexe.*

Le problème est formalisée sous forme canonique :

$$\left\{ \begin{array}{ll} \text{maximiser} & 3x_1 + 4x_2 \\ \text{avec les contraintes} & x_1 + 2x_2 \leq 12 \\ & x_1 + x_2 \leq 7 \\ & 2x_1 + x_2 \leq 12 \\ \text{et } x_1, x_2 & \geq 0 \end{array} \right.$$

Il est passé sous forme standard en introduisant les variables d'écart x_3, x_4 et x_5 .

$$\left\{ \begin{array}{ll} \text{maximiser} & z = 3x_1 + 4x_2 \\ \text{avec les contraintes} & x_3 = 12 - x_1 - 2x_2 \\ & x_4 = 7 - x_1 - x_2 \\ & x_5 = 12 - 2x_1 - x_2 \end{array} \right.$$

Nous pouvons maintenant résoudre ce programme par l'algorithme du simplexe.

Avec notre programme les solutions de base sont :

$$(\overline{x_1}, \overline{x_2}, \overline{x_3}, \overline{x_4}, \overline{x_5}) = (0, 0, 0, 12, 7, 12)$$

et la valeur de l'objectif $z = 0$.

La variable x_2 est la variable hors-base qui a le plus grand coefficient dans la fonction objectif, c'est elle dont la valeur est augmentée. Dans les contraintes nous avons :

$$\begin{aligned} x_2 > 6 & \Rightarrow x_3 < 0 \text{ contrainte la plus stricte} \\ x_2 > 7 & \Rightarrow x_4 < 0 \\ x_2 > 12 & \Rightarrow x_5 < 0 \end{aligned}$$

C'est la première contrainte qui est la plus stricte. Nous exprimons donc x_2 dans cette équation :

$$x_2 = 6 - \frac{1}{2}x_1 - \frac{1}{2}x_3$$

Ceci qui nous donne le programme suivant en remplaçant x_2 :

$$\left\{ \begin{array}{l} z = 3x_1 + 4(6 - \frac{1}{2}x_1 - \frac{1}{2}x_3) \\ x_2 = 6 - \frac{1}{2}x_1 - \frac{1}{2}x_3 \\ x_4 = 7 - x_1 - (6 - \frac{1}{2}x_1 - \frac{1}{2}x_3) \\ x_5 = 12 - 2x_1 - (6 - \frac{1}{2}x_1 - \frac{1}{2}x_3) \end{array} \right.$$

Et se simplifie en :

$$\left\{ \begin{array}{l} z = 24 + x_1 - 2x_3 \\ x_2 = 6 - \frac{1}{2}x_1 - \frac{1}{2}x_3 \\ x_4 = 1 - \frac{1}{2}x_1 + \frac{1}{2}x_3 \\ x_5 = 6 - \frac{3}{2}x_1 + \frac{1}{2}x_3 \end{array} \right.$$

Pour ce nouveau programme, la solution avec les variables hors-base nulles est :

$$(\overline{x_1}, \overline{x_2}, \overline{x_3}, \overline{x_4}, \overline{x_5}) = (0, 6, 0, 0, 1, 6)$$

Et la valeur objectif $z = 24$.

Cette fois c'est au tour de x_1 d'entrer en base. Les contraintes sont maintenant :

$$\begin{aligned} x_1 > 12 &\Rightarrow x_2 < 0 \\ x_1 > 2 &\Rightarrow x_4 < 0 \text{ contrainte la plus stricte} \\ x_1 > 4 &\Rightarrow x_5 < 0 \end{aligned}$$

C'est la seconde équations qui est la plus contraignante et qui donne l'expression de x_1 .

$$x_1 = 2 + x_3 - 2x_4$$

Ceci qui nous donne le programme suivant en remplaçant x_1 :

$$\begin{cases} z = 24 + (2 + x_3 - 2x_4) - 2x_4 \\ x_2 = 6 - \frac{1}{2}(2 + x_3 - 2x_4) - \frac{1}{2}x_3 \\ x_1 = 2 + x_3 - 2x_4 \\ x_5 = 6 - \frac{3}{2}(2 + x_3 - 2x_4) + \frac{1}{2}x_3 \end{cases}$$

Et se simplifie en :

$$\begin{cases} z = 26 + x_3 - 4x_4 \\ x_2 = 5 - x_3 + x_4 \\ x_1 = 2 + x_3 - 2x_4 \\ x_5 = 3 - x_3 + 3x_4 \end{cases}$$

Il n'y a plus de variable à coefficient positif dans l'équation, nous sommes donc arrivés au bout de la maximisation avec :

$$(\overline{x_1}, \overline{x_2}, \overline{x_3}, \overline{x_4}, \overline{x_5}) = (2, 5, 0, 0, 0, 3)$$

Et la valeur objectif $z = 26$.

Solution question 3.4 : *Donner la solution du problème d'optimisation.*

Nous confirmons bien, avec la résolution par le simplexe, la solution optimale trouvée avec la méthode graphique. Elle permet de réaliser 7 tablettes dont 2 avec plus d'amandes et 5 avec plus de noisettes, pour un gain potentiel de 26 €. Nous voyons également qu'il reste 30 grammes d'amandes qui ne sont pas consommés, à travers la dernière contrainte pour laquelle x_5 conserve une valeur résiduelle.